

Frame-Synchronous Filler ASR with a Frozen HuBERT-XL Encoder: A Controlled Comparison of Extended Training, Batch Scaling, and Train-Time Feature Masking

Date: 2026-07-12 · **Decode protocol:** `dump_decodes.py` (greedy frame argmax, cap = min(first `</s>`, duration ceiling `n_keep`), no CTC collapse, no LM) · **Hardware:** single RTX 6000 Ada per decode

Systems under test (all under `/speech/tomson/filler_asr/train_sa_pe_lm_head/runs/`):

ID	Run directory	Best ckpt
Baseline	<code>full960_sa8_pe_fps25_lin</code>	step 28,000 (epoch 26/30)
Variant A	<code>full960_sa8_pe_fps25_lin_100ep_bs32acc2</code>	step 40,000 (epoch 55/100)
Variant B	<code>full960_sa8_pe_fps25_lin_100ep_mask10-20x10_bs32acc2</code>	step 68,000 (epoch 62/100)

COMPARATIVE ANALYSIS REPORT · FILLER-ASR / SA8+PE HEAD · 2026-07-12

1. Executive Summary

All three systems share an identical inference-time architecture: a **frozen HuBERT-xlarge-ls960-ft encoder** (~1.27 B params, CTC head discarded) whose last hidden state is tapped, augmented with **fixed sinusoidal positional encodings**, passed through a trainable stack of **8 post-norm Transformer encoder layers** (d=1280, 16 heads, FFN 5120), and projected by a linear `lm_head` onto a 33-symbol character vocabulary that includes a special `<fill>` token. The training objective is **frame-synchronous character CE at 25 label frames/second**: the transcript's characters are placed one-per-frame from the start of the utterance, terminated by `</s>`, and all remaining frames inside the duration ceiling (`n_keep = ceil(dur × 25)`) are supervised as `<fill>`. Decoding is a per-frame argmax cut at the first `</s>` — there is no beam search, no CTC collapse, and no language model. Only the SA stack + `lm_head` (+ a mask embedding in Variant B) are trained; the encoder is never updated.

The three runs form an incremental ablation:

- **Baseline → Variant A** tests the training-budget hypothesis (H1): more epochs (30 → 100) and a larger effective batch (256 → 384) should improve generalization.
- **Variant A → Variant B** tests the masking hypothesis (H2): SpecAugment-style span masking of the tapped features (10-20 % of frames, 200 ms spans, learned `mask_embed`, applied before the PE add, train-time only) should force the SA stack to interpolate missing acoustic content from context and improve robustness, especially out of domain.

Findings. H1 is **rejected**: Variant A is nominally worse than the Baseline on all four test sets under the honest (capped) decode, and paired sign tests show no significant utterance-level change. Its large apparent gain on the training-time dev metric (20.5 % → 13.5 %) is shown to be an artifact of that metric's uncapped decoding. H2 is **confirmed**: Variant B improves over both Baseline and Variant A on every test set — significantly so on the two large sets (LibriSpeech test-clean $p \approx 1e-7$, L2-Arctic $p \approx 2e-30$, paired sign tests) — improves all 12 L2-Arctic speakers uniformly, and delivers its largest gains exactly where the architecture is weakest: long utterances (−7.9 pp WER absolute on >40-word LibriSpeech utterances vs. Baseline) and heavily accented speech. The characteristic failure mode of this architecture is not hallucination but **alignment smearing / tail collapse**, and masking measurably mitigates (without eliminating) it.

2. System Description and Experimental Configurations

2.1 Shared architecture and objective

```
16 kHz wav → HuBERT-xlarge (FROZEN, 48 layers, d=1280) → tap last_hidden_state (50 fps)
    → [Variant B, train only: mask 10–20 % of frames in 10-frame spans → mask_embed]
    → + sinusoidal PE
    → 8 × TransformerEncoderLayer (post-norm, 16 heads, FFN 5120, dropout 0.1) [TRAINABLE]
    → Linear 1280 → 33 [TRAINABLE]
loss: frame-wise CE against (s) c1 c2 ... cn {/s} <fill> ... <fill> (25 label fps, inside n_keep only)
```

The vocabulary is 33 tokens: 26 letters, `|` (word boundary), `'`, and specials `<s>` `</s>` `<pad>` `<unk>` `<fill>`. Because the target places characters at fixed positional frame indices (not acoustically aligned ones), the SA stack must learn both transcription and a positional re-alignment — a property that turns out to dominate the error analysis (§ 5).

2.2 Configuration deltas (recovered from checkpoint `args` and launch scripts)

Hyper-parameter	Baseline	Variant A	Variant B
Epochs (planned)	30	100	100
Best ckpt @ step / epoch	28 k / 26	40 k / 55	68 k / 62
GPUs × per-GPU batch × grad-accum	2 × 32 × 4	6 × 32 × 2	4 × 32 × 2
Effective batch	256	384	256
LR / schedule / warmup	2e-4 / linear / 2000	same	same
Weight decay	0.005	same	same
Feature masking	—	—	p ~ U[0.10, 0.20], span 10 fr (200 ms), learned embed
Trainable params	SA + lm_head	same	same + mask_embed (1280)
Train data	LibriSpeech 960 h	same	same
Dev selection	dev-clean[:500], uncapped WER	same	same

Two provenance notes recovered during this analysis: (i) the launch script `train_sa8_mask_100ep_4gpu.sh` claims `accum = 3` for "384 batch parity" in its header comment, but the checkpoint records `grad_accum=2` — Variant B actually trained at **effective batch 256**, i.e. batch-matched to the Baseline, not to Variant A; (ii) effective batches were independently confirmed from optimizer steps-per-epoch (281 k train utts / 256 ≈ 1.1 k steps/ep, observed 1077–1097 for Baseline/B; /384 ≈ 732, observed 727 for A).

2.3 Test sets

Set	Manifest	n	Nature
libri-clean	<code>data/test_clean.jsonl</code>	2,620	in-domain read speech (LibriSpeech test-clean)
seedTTS-prompts	<code>testsets/en/seedtts_en_prompts.jsonl</code>	666	real Common-Voice speech (Seed-TTS-eval reference prompts)
seedTTS-pairs	<code>testsets/en/seedtts_en_pairs.jsonl</code>	1,088	synthetic speech (TTS outputs of the Seed-TTS eval protocol)
L2-Arctic (full)	<code>testsets/l2arctic_data/l2arctic_full.jsonl</code>	13,292	non-native accented English, 12 speakers (Arabic, Mandarin, Hindi, Korean, Spanish, Vietnamese L1s), 13.4 h

All audio 16 kHz mono; references normalized identically to training (lowercase, punctuation stripped). All 17,666 utterance decodes per model are line-aligned across the three models (verified reference-identical), enabling paired analysis.

3. Consolidated Quantitative Results

3.1 Primary metrics (capped decode — the honest protocol)

WER (%) — best per set in **bold**:

Set	n	Baseline	Variant A	Variant B (mask)	Δ B–Base	Δ B–A
libri-clean	2,620	12.04	12.54	9.94	−2.10	−2.60
seedTTS-prompts	666	11.33	11.74	10.96	−0.37	−0.78
seedTTS-pairs	1,088	9.63	9.91	9.25	−0.38	−0.66
L2-Arctic full	13,292	24.88	24.93	23.11	−1.77	−1.82

CER (%) / SER (%):

Set	Baseline	Variant A	Variant B
libri-clean	6.13 / 34.96	5.66 / 35.99	4.65 / 33.66
seedTTS-prompts	3.29 / 51.05	3.32 / 51.05	3.18 / 51.80
seedTTS-pairs	2.80 / 45.13	2.86 / 45.96	2.67 / 43.38
L2-Arctic full	10.24 / 66.58	10.41 / 66.42	9.58 / 63.84

Variant B is best on 11 of 12 set×metric cells (the exception: prompts SER, +0.75 pp — B wins fewer perfect sentences there while making fewer errors overall).

3.2 Statistical reliability (paired, utterance-level)

Sign tests on per-utterance WER (ties dropped):

Comparison	libri	prompts	pairs	L2-Arctic
B better / worse than A	427 / 269, p=2.1e-9	108 / 88, p=0.15	154 / 127, p=0.11	3006 / 2301, p=3.8e-22
B better / worse than Base	396 / 260, p=1.1e-7	95 / 88, p=0.61	129 / 120, p=0.57	3125 / 2283, p=2.4e-30
A better / worse than Base	346 / 368, p=0.41	94 / 95, p=0.94	145 / 144, p=0.95	2766 / 2630, p=0.06

Variant B's advantage is decisive on the two large sets and directionally consistent (but not individually significant at $n \leq 1,088$) on the two SeedTTS sets. Variant A is statistically indistinguishable from the Baseline everywhere despite 2.3× the training budget.

3.3 The dev-metric mismatch (why Variant A looked like a win during training)

Training-time model selection uses dev-clean[:500] with an **uncapped** batch decode: `batch_decode` runs over all padded frames, so any garbage emitted after `</s>/n_keep` counts as insertions (a known issue, previously analyzed in `eval_analysis/`).

Metric	Baseline	Variant A	Variant B
Dev WER (train-time, uncapped)	20.52	13.50	8.52
Test-clean WER (capped)	12.04	12.54	9.94

The Baseline→A "improvement" of −7 pp dev WER coexists with a +0.5 pp degradation in capped test WER: extended training mostly taught Variant A to emit cleaner `<fill>/</s>` tails (which only the uncapped metric rewards), not better transcriptions. Direct leak measurements corroborate the tail-garbage story: Variant B's final checkpoint leaks post-`</s>` garbage on 236/2,620 test-clean utterances (uncapped 11.52 % vs. capped 10.06 % under `infer.py`'s `</s>`-only cap), while an earlier-step Baseline-family checkpoint measured 366/2,620 leaked with a +6.8 pp uncapped penalty.

Implication: uncapped dev WER is not a valid model-selection signal for this architecture; all conclusions in this report use capped decoding.

3.4 L2-Arctic per-speaker breakdown (WER)

Speaker (L1)	Baseline	Variant A	Variant B	Δ B–Base
ABA (Arabic)	19.28	19.91	18.94	−0.34
ASI (Hindi)	13.68	14.41	11.81	−1.87
BWC (Mandarin)	34.19	33.17	30.76	−3.43
EBVS (Spanish)	39.22	39.87	38.17	−1.05
HJK (Korean)	14.85	13.55	12.16	−2.69
HKK (Korean)	24.38	24.41	23.74	−0.64
HQTV (Vietnamese)	47.28	48.33	45.65	−1.63
LXC (Mandarin)	29.94	29.48	27.19	−2.75
MBMPS (Spanish)	13.17	13.61	12.98	−0.19
PNV (Vietnamese)	16.99	17.23	16.07	−0.92
SKA (Arabic)	32.35	32.84	29.90	−2.45
SVBI (Hindi)	15.83	15.14	12.63	−3.20

Variant B wins **12/12 speakers** against both other systems — a uniformity that a noise explanation cannot produce. Variant A vs. Baseline is mixed (better on 4, worse on 8). Speaker difficulty spans 12–47 % WER; masking helps across the entire range, with the largest absolute gains on mid-to-hard speakers (BWC −3.4, SVBI −3.2, LXC −2.8).

4. Error Decomposition and Transcription Completeness

4.1 Substitution / insertion / deletion rates (% of ref words)

Set	Model	Sub	Ins	Del	hyp/ref length ratio	exact-match utts	utts WER $\geq$ 50 %
libri	Base	6.90	0.16	4.98	0.952	1,705	100
libri	A	7.36	0.16	5.02	0.951	1,677	115
libri	B	5.99	0.21	3.73	0.965	1,738	82
L2-Arctic	Base	20.62	0.95	3.31	0.976	4,447	2,825
L2-Arctic	A	19.35	0.95	4.63	0.963	4,466	2,762
L2-Arctic	B	18.80	1.18	3.14	0.980	4,807	2,550

Three structural observations:

- Insertions are near-zero for every system (≤ 1.2 %).** The duration-ceiling design (`n_keep`) plus `</s>` capping makes classic open-ended hallucination — fluent invented content — architecturally impossible to sustain: the model cannot emit more than ~ 2 chars per real frame of audio. The dominant errors are substitutions and deletions, i.e. **corruption and truncation, not confabulation**.
- Variant A trades substitutions for deletions on hard audio.** On L2-Arctic its sub rate is 1.3 pp better than Baseline but its deletion rate is 1.3 pp worse (4.63 % vs 3.31 %), and its length ratio is the lowest everywhere (0.951–0.963). Extended training made A more conservative: when

uncertain it drops words. This is precisely the behavior that inflates its dev-metric advantage (shorter output = less tail garbage) while degrading true completeness. A also produced the only fully **empty hypothesis** in the entire evaluation (1 utt, L2-Arctic).

3. **Variant B is simultaneously more accurate and more complete:** lowest sub and del on all sets, length ratio closest to 1.0 (0.965-0.999), most exact matches, fewest catastrophic (≥ 50 % WER) utterances — at the cost of a marginal insertion uptick (+0.05-0.23 pp).

4.2 WER by utterance length (LibriSpeech test-clean)

Ref length	n	Baseline	Variant A	Variant B
1-10 words	733	3.75	4.55	4.29
11-20 words	902	4.36	4.64	3.97
21-40 words	751	6.27	8.13	5.71
>40 words	234	34.52	32.72	26.60

This table localizes the architecture's core weakness: WER is a healthy 4-6 % up to 40 words, then **explodes ~6×** on longer utterances. The 234 longest utterances contribute a large share of total error. The gradient across systems is equally clear — masking cuts the long-utterance WER by 7.9 pp absolute (23 % relative) over the Baseline, consistent with H2's mechanism: a model trained to bridge 200 ms holes maintains a coherent positional alignment further into the utterance. (On short utterances the Baseline remains best by ~0.5-0.8 pp — masking is a regularizer and slightly blurs easy cases; L2-Arctic, whose utterances are all ≤ 20 words, shows B winning both buckets.)

5. Qualitative Analysis: Failure Taxonomy with Decode Evidence

All examples below are verbatim from the line-aligned `*.hyp.txt` / `*.decodes.txt` artifacts; keys allow direct lookup.

5.1 Failure mode 1 — alignment smearing (the dominant catastrophic error)

Because targets are positional (25 chars/s from $t=0$), a model whose internal alignment drifts writes each character across several frames or at shifted slots; after per-frame argmax with **no CTC collapse**, this surfaces as stuttered/duplicated letters, not as wrong words. It is bistable — an utterance either decodes cleanly or degenerates almost entirely:

61-70968-0062 (libri) — REF: `ay and show you some pretty tricks` - A: `ayeanddshowyyuussoeprtty trrcks` (WER 1.00) - Baseline: `ay and show you some pretty tricks` (0.00) - B: `ay and show you some pretty tricks` (0.00)

SVBI_arctic_b0302 (L2, Hindi L1) — REF: `i pulled suddenly with all my might` - A: `pu lel dudedln yiti h a l mimhthh` (1.14) — B and Baseline: perfect.

The effect is not monotone in favor of B; masking shifts **which** utterances collapse. B's regressions are the same phenomenon in reverse:

SVBI_arctic_b0296 — REF: `at times i wondered where sir archibald got his style` - Baseline: perfect - A: perfect - B: `ieesieis iwweeweheherri iarhhichdbalt ht ityssy` (WER 1.00)

5105-28241-0017 (libri, 12.8 s) — REF: `...i should expect to find a depth of two or three hundred fathoms...` - A: perfect (0.00) - Baseline: near-perfect - B: `after pondering a whil he aaid i ww wwrre fatther wwayii should epec ...` (0.79)

The net accounting, however, is unambiguous: B collapses fewer utterances than it repairs on every set (e.g. L2-Arctic 3,006 repaired vs 2,301 regressed vs A; libri catastrophic count 82 vs A's 115), so masking reduces the smearing rate ~20-30 % without eliminating its stochastic character.

5.2 Failure mode 2 — long-utterance tail collapse (the closest thing to hallucination)

On utterances approaching/exceeding the ~10 s training crop (`sample_max_sec=10`), all systems eventually lose alignment and the tail degenerates into low-entropy character loops — perseverative garbage, not fluent confabulation:

7729-102255-0032 (libri, **20.3 s**, T=1013 frames) — REF ends: ...and provided a free dinner in honor of the occasion - A (WER 0.18): transcribes 45 of 50 words then truncates: ...to the public and privid ffreeeee - Baseline: correct for ~40 words then: ...to the publiccaarrvvv f n ooooooooo - B (WER 0.90): loses alignment earlier on this utterance and emits smeared text, ending ...rivde eeeeeerooooooooo

2094-142345-0010 (libri, 78-word ref): all three collapse mid-utterance; B survives visibly longest (...for the oak table was usually turnedd p likea screee and waa mmre ffo oraamen than ffr uue... before degenerating), Baseline degenerates almost immediately (hetty surrrll oftettotoott pppootonny...).

This mode explains nearly all of the >40-word bucket's 27-35 % WER and is the highest-leverage remaining problem (candidate fixes: chunked/streaming decode, long-crop fine-tuning, or monotonic alignment constraints).

5.3 Transcription completeness, `</s>` discipline, and stripping robustness

- The strip-variant sweep (`eos` vs `fillrun3` vs `eos_or_fillrun3` cuts) changes WER by **≤0.03 pp for every model on every set** — i.e. a missing `</s>` inside the duration ceiling is rare for all systems; the token-emission protocol is learned reliably. (Best-variant selection flips between `eos` and `eos_or_fillrun3` only on ties.)
- Post-`</s>` leakage within `n_keep` exists but is metric-invisible under the capped protocol (§ 3.3). Variant B leaks on 9 % of test-clean utts vs ~14 % for an earlier-generation checkpoint — masking also tightened tail discipline.
- Variant A's completeness deficit is systematic (length ratio 0.951-0.963, deletion-dominant profile, e.g. dropping sentence-initial words: `SVBI_arctic_a0040` REF `i suppose you wonder...` → A `suppose you wonder...`), and it is the only system to output an empty hypothesis.

5.4 Edge cases all systems share

- **Proper nouns / rare words** (no LM to rescue a character model): `1089-134691-0024` REF `stephanos dedalos` → Base `stefhino's dead loss` / A `steppano's dead loss` / B `steffano's dead loss` — phonetically faithful, lexically hopeless, in all three.
- **Accent-driven confusions persist post-masking**: BWC/LXC `"my name's ferguson"` → `my name is fer golssn / fergunsar` for every model (contraction expansion + name).
- **Synthetic-speech artifacts** (seedTTS-pairs): TTS outputs are easier than real prompts for all models (9.3-9.9 % vs 11.0-11.7 %) — TTS over-articulation aids a frame-synchronous reader — and B's win there (-0.66 vs A) shows the masking benefit is not specific to natural-speech noise.
- **Prompts SER anomaly**: B fixes hard utterances (31 vs 43/47 catastrophic) but perfects marginally fewer easy ones (322 vs 326 exact) — the regularization trade visible at set scale.

6. Discussion

Why extended training + larger batch (A) failed. Variant A optimized the objective it was given: with $2.3\times$ more updates it learned cleaner `<fill>` tails and more confident (hence more deletion-prone) outputs — exactly the behaviors the uncapped dev-selection metric rewards — while its true transcription quality drifted slightly below the 30-epoch Baseline (over-fitting of the alignment behavior to the 960 h training distribution is consistent with its worst-in-class smearing rate on libri, 115 catastrophic utts). This is a textbook **metric-objective mismatch**, and it silently consumed a $6\text{-GPU} \times \sim 55\text{-epoch}$ training budget.

Why masking (B) worked. Span-masking 10-20 % of the tapped features creates exactly the supervision the frame-positional objective otherwise lacks: frames whose content is absent but whose position must still emit the right character force the SA stack to maintain a robust internal alignment driven by context rather than by local acoustics. The signature of that mechanism is where B wins: long utterances (-7.9 pp), accented speech (12/12 speakers), degraded/unusual audio — and simultaneously lower substitutions **and** deletions with near-unchanged insertions. A pure regularizer would trade these off; an alignment-stabilizer improves both, which is what the data shows. Note that B achieved this at effective batch 256 (Baseline parity), so the gain is attributable to masking alone, not batch effects.

Threats to validity. (i) Single seed per configuration; the mask-regression examples show per-utterance bistability, so per-set deltas <0.8 pp (both SeedTTS sets) should be treated as directional evidence only — the sign tests there do not reach significance. (ii) Checkpoints are best-on-dev under a flawed dev metric; a capped dev metric might select different (possibly better) checkpoints, likely understating all three systems. (iii) Variant B trained $\sim 1.7\times$ more optimizer steps than A at its best checkpoint (68 k vs 40 k), though A had plateaued on dev by 40 k within a 73 k-step budget; the batch-256 parity with Baseline partially controls the A $\leftrightarrow$ B step confound but does not eliminate it. (iv) Baseline $\leftrightarrow$ A differ in two factors (epochs and batch), so H1's rejection applies to the bundle, not to each factor separately.

7. Conclusion

Under a leak-proof, duration-capped decoding protocol across 17,666 test utterances spanning in-domain read speech, real conversational prompts, synthetic TTS audio, and non-native accented English:

- 1. The extended-training/batch-scaling update (Variant A) provided no measurable benefit and a small consistent degradation** — worse aggregate WER on 4/4 test sets ($+0.05$ to $+0.50$ pp), statistically indistinguishable at the utterance level (all $p \geq 0.06$), with a systematic deletion/truncation bias. Its apparent 7-point dev-WER gain was an artifact of the uncapped training-time metric and should be disregarded.
 - 2. Train-time feature masking (Variant B) provided a definitive, measurable benefit:** best WER, CER on all four sets (-0.37 to -2.10 pp WER vs Baseline; -0.66 to -2.60 vs Variant A), significant paired improvements on both large sets ($p \leq 1.1e-7$), uniform wins across all 12 L2-Arctic speakers, a 23 % relative WER reduction on the architecture's worst regime (>40 -word utterances), and improved completeness (fewest deletions, length ratio closest to unity) at negligible insertion cost. **The masking update is unambiguously the correct configuration to carry forward.**
 - 3. The remaining performance frontier is not acoustic modeling but positional-alignment stability:** stochastic per-utterance smearing and deterministic long-utterance tail collapse dominate residual error. Recommended next steps: capped-WER dev selection, long-crop or curriculum training beyond 10 s, and decode-time chunking — each targets the failure modes documented in § 5 directly.
-

Appendix A. Artifact Map

```

runs/full960_sa8_pe_fps25_lin/          # Baseline
  libri_clean/test_clean.*              # n=2620 WER 12.04
  seedTTS/seedtts_en_{prompts,pairs}.*  # 11.33 / 9.63
  l2arctic/l2arctic_full.*              # 24.88
runs/full960_sa8_pe_fps25_lin_100ep_bs32acc2/ # Variant A
  librispeech_test_clean.*              # 12.54
  seedTTS/... l2arctic/...              # 11.74 / 9.91 / 24.93
runs/full960_sa8_pe_fps25_lin_100ep_mask10-20x10_bs32acc2/ # Variant B
  test_clean.*                          # 9.94
  seedTTS/... l2arctic/...              # 10.96 / 9.25 / 23.11

```

Each set ships five files: `.decodes.txt` (per-utt RAW/NKEEP/HYP/REF chains), line-aligned `.hyp.txt/.ref.txt`, `.dump.summary` (JSON aggregate), `.strip_variants.summary` (cut-strategy sweep). Decoder: `dump_decodes.py` (mask-checkpoint support added 2026-07-12); per-utterance capped/uncapped comparison: `infer.py`.

Appendix B. Reproduction

```

cd /speech/tomson/filler_asr/train_sa_pe_lm_head
RUN=<run_dir_name> GPU=<free_gpu> OUT_DIR=<run>/<set_subdir> \
MANIFEST=<manifest.jsonl> bash dump_decodes.sh

```